

Recapitulare SO

MINI “TABEL DE ASOCIERI” (super util la invatat)

- CPU management -> scheduling (Gantt, RR, SJF)
- Memorie -> paging / page table / TLB / page fault
- I/O -> interrupt / DMA / driver
- Fisiere -> FAT / indexare / alocare blocuri
- Protectie -> user vs kernel + valid/invalid + permisiuni
- Resurse + concurenta -> deadlock + mutex/semafor

Drivere - este un program (parte din sistemul de operare) care face legatura intre hardware si sistemul de operare.

Cum functioneaza un driver (pas cu pas)

EXEMPLU: scriere pe disc

Programul apeleaza:

`write(fd, buffer, size)`

1. Se face **system call**
2. CPU trece din **user mode in kernel mode**
3. Kernelul identifica:
 - `fd` → fisier
 - fisier → sistem de fisiere
 - sistem de fisiere → driver de disc
4. Kernelul apeleaza **driverul**
5. Driverul:
 - pregateste comenzi pentru controller
 - foloseste DMA sau I/O
 - porneste operatia
6. Hardware-ul executa operatia
7. Dispozitivul genereaza **interrupt**
8. Driverul trateaza interrupt-ul
9. Controlul revine aplicatiei

Aplicatia NU vede nimic din pasii 4–9.

Tipuri de drivere (trebuie sa le stii)

1) Device driver

- pentru hardware:
 - disc
 - tastatura
 - mouse
 - placa de retea
 - imprimanta

2) Driver de sistem de fisiere

- implementeaza:
 - FAT
 - ext4
 - NTFS
- traduce operatii:
 - open, read, write → blocuri pe disc

3) Driver de retea

- TCP/IP
- placi de retea
- socket-uri

Drivere si I/O (legatura importanta)

Polling

- driverul verifica constant daca dispozitivul e gata
- CPU ocupat inutil

Interrupts

- dispozitivul anunta cand a terminat
- driverul trateaza interrupt-ul
- mai eficient

DMA (Direct Memory Access)

- driverul:
 - configureaza transferul
 - hardware-ul muta date direct in RAM

- CPU NU muta datele byte cu byte

La examen:

DMA = mai rapid, mai putin CPU

Drivere si fisiere (asociere foarte importanta)

In UNIX:

- **totul este fisier**

Exemple:

- /dev/sda → disc
- /dev/tty → terminal
- /dev/null → device special

Cand faci:

`read("/dev/sda")`

→ kernelul apeleaza **driverul de disc**

USER MODE VS KERNEL MODE

CPU poate rula in doua moduri:

User mode

- ruleaza aplicatiile
- **NU au voie** instructiuni periculoase:
 - acces direct I/O
 - modificare pagetable
 - oprire intreruperi
 - acces la memoria kernelului

Kernel mode

- ruleaza OS-ul (kernel)
- are voie **tot**: I/O, memorie, drivere, scheduling etc.

Cum treci din user in kernel?

Prin:

- **system call** (ex: open, read, write)

- **interrupt** (tastatura, disk, timer)
- **exception / trap** (ex: page fault)

I/O e relevant aici: intreruperile si trap-urile sunt mecanisme standard prin care CPU intra in kernel ca sa trateze evenimentul.

SYSTEM CALL

System call = “functie” prin care o aplicatie cere kernelului sa faca ceva privilegiat.

Exemple clasice:

- `open()`, `read()`, `write()`, `close()` (lucru cu fisiere)
- `fork()`, `execve()` (proces)
- `mmap()` (memorie)
- `kill()` (semnale)

Laburile explica concret ca syscalls sunt cele din manual sectiunea 2 si cum se folosesc `read/write/open/close`.

De ce exista

Pentru ca aplicatia:

- nu are voie direct la hardware
- kernelul trebuie sa verifice drepturi, parametri, protectie
- kernelul sa poata administra resursele corect

Cum arata in practica

Vrei sa scrii pe ecran.

Aplicatia face:

- `write(1, "Hi", 2)`

Ce se intampla logic:

1. aplicatia e in user mode
2. face system call -> trap
3. CPU intra in kernel mode
4. kernelul verifica parametrii + accesul la `fd=1`
5. kernelul apeleaza driverul de consola

6. revine in user mode

In lab 1 se vede exact ideea asta (executabilul apeleaza write in kernel).

INTREBARI TIPICE DE EXAMEN (si raspunsurile scurte “de 10”)

1. “De ce avem user mode si kernel mode?”
 - Ca sa protejam sistemul: aplicatiile nu pot executa instructiuni privilegiate, accesul se face controlat prin system calls.
2. “Ce este un system call?”
 - Interfata prin care un program cere kernelului servicii ce necesita privilegii (I/O, procese, memorie).
3. “Ce e o resursa? Da exemple.”
 - Orice componenta limitata administrata de OS: CPU, memorie, device, lock-uri, fisiere, etc.

Procese

Un proces are, in mod tipic:

A) Spatiu de adrese (memorie)

- **Code/Text:** instructiunile programului
- **Data:** variabile globale/statice initializate
- **BSS:** globale/statice neinitializate (pornesc 0)
- **Heap:** memorie dinamica (malloc/new) – creste “in sus”
- **Stack:** apeluri de functii, variabile locale – creste “in jos”

Asociere pentru memorie:

- cand auzi “heap/stack” -> te gandesti la “spatiu de adrese” al procesului
- cand auzi “paging / page table” -> e modul cum OS mapeaza spatiul asta in RAM

B) Context CPU (starea de executie)

- **registre** (inclusiv PC/IP – program counter)
- **SP** (stack pointer)
- **PSW/flags** (stare CPU)
- “unde era” procesul in cod in momentul intreruperii

C) Resurse OS

- fisiere deschise (file descriptors)
- semnale / handler (daca le ai in materie)
- drepturi (UID/GID), permisiuni

- info de scheduling (prioritate, quantum etc.)

Ce campuri sunt obligatorii

Cand profesorul zice “ce are PCB-ul?”, tu scrii (macar):

1. **PID** (process id)
2. **stare** (new/ready/running/blocked/terminated)
3. **registre salvate** (context)
4. **info scheduling** (prioritate, timp ramas, pointer in coada ready)
5. **info memorie**:
 - pointer la **page table** / info segmentare (depinde de capitol)
6. **fisiere deschise** (tabela de descriptori)
7. **relatii**:
 - PPID (parinte), copii, etc.

Legatura cu laburi:

- in laburile de procese se vede clar ca fork() creeaza copil cu PID diferit si OS gestioneaza relatia parinte-copil + wait() pentru sincronizare

Starile procesului (modelul clasic)

Starile si semnificatia lor

- **new**: procesul e creat, OS ii pregateste PCB, memorie etc.
- **ready**: are tot ce ii trebuie, asteapta CPU (in coada ready)
- **running**: ruleaza efectiv pe CPU
- **waiting / blocked**: asteapta un eveniment (I/O, semafor, lock, sleep)
- **terminated**: s-a terminat (poate ramane “zombie” pana il colecteaza parintele cu wait)

Tranzitii

- ready -> running: **scheduler** il alege
- running -> ready: expira cuantumul (timer interrupt) sau e preemptat
- running -> blocked: cere I/O / asteapta lock / wait
- blocked -> ready: I/O terminat / lock disponibil / eveniment produs
- running -> terminated: exit

Context switch (schimbare de context)

Context switch = OS opreste procesul curent si porneste alt proces, salvand si restaurand starea CPU.

Cand se intampla (cauze tipice)

1. **timer interrupt** (expira quantum)
2. procesul face **I/O** (se blocheaza)
3. procesul se termina
4. procesul e preemptat (vine unul cu prioritate mai mare)

Pasii: Cand OS face context switch:

Pas 1: salveaza contextul procesului A

- salveaza registre, PC, SP, flags in PCB(A)

Pas 2: actualizeaza starea lui A

- daca i-a expirat quantum: A devine **ready**
- daca asteapta I/O: A devine **blocked**

Pas 3: alege procesul B

- scheduler scoate din coada ready procesul ales

Pas 4: restaureaza contextul lui B

- incarca registre/PC/SP din PCB(B)

Pas 5: ruleaza B

- CPU continua executia exact de unde ramasese B

EX 1 (teorie, 10p)

Cerinta: “Defineste procesul si enumera ce contine PCB.”

Rezolvare (model de scris):

- Proces = program in executie + spatiu de adrese + resurse + context.

- PCB contine: PID, stare, registre salvate, info scheduling, info memorie (page table), fisiere deschise, relatii parinte/copil.

EX 2

Cerinta: “Ai codul de mai jos. Cate procese se creeaza si ce afiseaza?”

```
printf("A\n");

pid_t p = fork();

if (p == 0) {

    printf("C\n");

} else {

    wait(NULL);

    printf("P\n");

}
```

```
printf("X\n");
```

- copil (p == 0)
so-lab-4
- 3. Copil:
 - afiseaza C
 - apoi afiseaza X
 - se termina
- 4. Parinte:
 - wait(NULL) asteapta copilul sa termine
so-lab-4
 - afiseaza P
 - afiseaza X

Ordine posibila a output-ului:

- A este mereu primul
 - datorita wait, parintele afiseaza P si X doar dupa ce copilul a terminat
- Deci o varianta tipica:

A C X P

Rezolvare pas cu pas

1. Initial: 1 proces afiseaza A
2. fork() -> devin 2 procese:
 - parinte (p > 0)

Cerinta: “Explica traseul starilor pentru un proces care ruleaza, cere I/O, apoi revine si continua.”

Rezolvare

- ready -> running (scheduler)
- running -> blocked (cere I/O)
- blocked -> ready (I/O terminat)
- ready -> running (scheduler iar)

Unde apare context switch?

- cand trece running->blocked (CPU trebuie dat altuia)
- cand alt proces devine running
- cand revine si e planificat iar

EX 3

Cerinta: “De ce thread-urile sunt mai rapide decat procesele? Da exemplu.”

Rezolvare

- thread-urile impart acelasi spatiu de adrese, deci creare/switch e mai ieftin decat la procese.
- exemplu: calculezi produsul de matrice cu cate un thread pe element (clasic in laborator), iar rezultatul se combina usor pentru ca memoria e comuna.

System calls IMPORTANTI

fork()

- creeaza proces copil
- returneaza:
 - 0 in copil
 - PID copil in parinte
- foloseste Copy-on-Write

exec / execve

- inlocuieste complet procesul curent cu alt program
- PID ramane acelasi
- nu revine daca reuseste

wait / waitpid

- sincronizare parinte-copil
- wait asteapta un copil oarecare
- waitpid asteapta un copil specific

exit

- termina procesul
- trimite status parintelui
- elibereaza resursele

open / read / write / close

- acces fisiere si device-uri
- read poate bloca
- write declanseaza I/O

kill

- trimite un semnal unui proces
- nu inseamna neaparat „omoara”

System call-urile permit unui program din user mode sa solicite kernelului executia unor operatii privilegiate precum I/O, creare de procese sau managementul memoriei.

– PROCES ZOMBIE SI ORPHAN

Zombie = proces terminat al carui parinte este inca activ si nu a apelat wait().

- NU ruleaza
- NU consuma CPU
- ocupa intrare in tabela de procese
- dispare cand parintele face wait sau moare

Cand apare:

- copilul moare
- parintele nu face wait

Orphan = proces activ al carui parinte s-a terminat.

- procesul ruleaza normal
- este adoptat de init (PID 1)
- init va face wait pentru el

Cand apare:

- parintele moare inaintea copilului

Schema de asociere

- fork → creeaza copil
- exec → schimba programul
- wait → colecteaza copil
- fara wait → zombie
- parinte moare → orphan
- orphan → adoptat de init

MINI TABEL DE ASOCIERI

- fork → procese
- exec → program nou
- wait → sincronizare parinte-copil
- open/read/write → fisiere si device
- pipe → comunicare IPC
- kill/signal → semnale
- pthread_* → thread-uri
- mutex/semafor → sincronizare

INTREBARI TIPICE DE EXAMEN (raspunsuri scurte)

1. **Ce face fork?**
Creeaza un proces copil; returneaza 0 in copil si PID copil in parinte.
2. **De ce fork + exec?**
Ca parintele sa ramana, iar copilul sa devina alt program.
3. **Ce se intampla fara wait?**
Copilul devine zombie pana e colectat.
4. **Ce se intampla dupa exec daca a reusit?**
Codul urmator nu se mai executa.

Thread

Ce partajeaza thread-urile vs ce NU partajeaza

Thread-urile din acelasi proces PARTAJEAZA:

- **codul** (text segment)
- **datele globale/statice**
- **heap** (malloc/new)
- **fisierele deschise** (file descriptors)
- **spatiul de adrese** (aceeasi memorie virtuala)

Thread-urile NU partajeaza:

- **stack-ul** (fiecare thread are stack separat)
- **registrele** (fiecare thread are propriul context CPU: PC, SP, regs)
- (adesea) **TLS** (thread-local storage) – idee

De ce conteaza?

- pentru ca daca ai variabile pe heap/global, doua thread-uri pot scrie simultan -> race condition
- daca ai variabile locale pe stack, fiecare thread are copia lui -> mai sigur

Mini-exemplu asociere:

- `int x;` global -> partajat -> necesita mutex daca e modificat de mai multe thread-uri
- `int y;` local intr-o functie -> e pe stack -> fiecare thread are alt y

Concurenta $\neq$ paralelism

- **Concurenta** = mai multe thread-uri *progreseaza* in timp (nu conteaza cum)
- **Paralelism** = ruleaza *simultan* pe mai multe core-uri

EXEMPLU:

Ai:

- 4 thread-uri: T1, T2, T3, T4
- 2 core-uri

Ce face OS-ul:

core 1: T1 $\leftrightarrow$ T3

core 2: T2 $\leftrightarrow$ T4

De ce thread-urile sunt “mai rapide” decat procesele

Thread-urile sunt mai “ieftine” pentru ca:

- nu creezi spatiu de adrese separat (deja exista)
- context switch intre thread-uri din acelasi proces e mai simplu decat intre procese (de obicei)

Dar: mai rapid nu inseamna automat mai bun, pentru ca vine cu:

- risc de race condition
- nevoie de sincronizare (mutex/semafor)
- debug mai greu

Avantaje

- comunicare rapida (memorie comuna)
- creare si switch mai ieftin decat proces
- ideal pentru:
 - servere (multi clienti)
 - aplicatii UI (un thread UI + worker threads)
 - paralelizare calcule (split & merge)

Dezavantaje

- buguri greu de prins (race condition)
- deadlock daca folosesti lock-uri prost
- un bug intr-un thread poate strica tot procesul (memorie comuna)

Asociere:

Procesele = izolare, siguranta

Thread-urile = viteza, dar risc

Cum creezi thread-uri in POSIX

- pthread_create creeaza thread
- pthread_join asteapta thread (echivalent conceptual cu wait la procese)

Schelet minim

```
#include <pthread.h>

void* worker(void* arg) {
    // cod thread
    return NULL;
}

int main() {
    pthread_t t;
    pthread_create(&t, NULL, worker, NULL);
    pthread_join(t, NULL);
    return 0;
}
```

Cand trebuie sincronizare (mutex)

Daca 2 thread-uri ating aceeași variabilă **partajată** (global/heap) și cel puțin unul scrie -> ai nevoie de sincronizare.

Exemplu race condition

```
int cnt = 0;

void* inc(void* arg){
    for(int i=0;i<100000;i++) cnt++;
    return NULL;
}
```

Dacă rulezi 2 thread-uri cu inc, rezultatul poate fi < 200000.

De ce? cnt++ nu e atomic:

- citire cnt
 - +1
 - scriere înapoi
- Intercalarea produce pierderi.

Fix cu mutex

```
pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;

void* inc(void* arg){
    for(int i=0;i<100000;i++){
        pthread_mutex_lock(&m);
        cnt++;
        pthread_mutex_unlock(&m);
    }
    return NULL;
}
```

Data parallelism = aceeași funcție, date diferite

Task parallelism = funcții diferite, aceleași date

Legea lui Amdahl – explicație

Viteza maxima pe care o poti obtine din paralelizare este limitata de partea SERIALA a programului.

Cu alte cuvinte:

- chiar daca ai foarte multe core-uri,
- **partea care nu poate fi paralelizata te tine pe loc.**

$$speedup \leq \frac{1}{S + \frac{(1-S)}{N}}$$

Unde:

- **S** = fractia **seriala** a programului (0...1)
- **1 - S** = fractia **paralela**
- **N** = numarul de core-uri
- **speedup** = de cate ori e mai rapid programul

Cum se citeste formula

- **S** sta singur → **nu se imparte**, nu se accelereaza
- **(1-S)/N** → doar partea paralela beneficiaza de N core-uri
- suma din numitor = **timpul total relativ**

Daca **S** e mare, speedup-ul e mic, oricate core-uri ai.

Exemplu numeric

Date

- 25% serial → **S = 0.25**
- 75% paralel
- **N = 2** core-uri

$$speedup = \frac{1}{0.25 + \frac{0.75}{2}} = \frac{1}{0.25 + 0.375} = \frac{1}{0.625} \approx 1.6$$

Ce se intampla daca N → infinit

Daca ai “infinit” de core-uri:

$$\lim_{N \rightarrow \infty} \text{speedup} = \frac{1}{S}$$

Exemple rapide:

- $S = 0.5 \rightarrow \text{speedup max} = 2$
- $S = 0.2 \rightarrow \text{speedup max} = 5$
- $S = 0.1 \rightarrow \text{speedup max} = 10$

Modele de implementare thread-uri (1-1, m-1, m-m)

Asta e intrebare de “concept”, nu de cod.

A) Many-to-one (M:1)

- multe user threads pe un singur kernel thread
- avantaje: foarte rapid de creat
- dezavantaje:
 - daca unul blocheaza (syscall blocking), blocheaza tot
 - NU ai paralelism real pe multicore

B) One-to-one (1:1)

- fiecare user thread = kernel thread
- avantaje:
 - paralelism real
 - blocking afecteaza doar thread-ul
- dezavantaje:
 - overhead mai mare (kernel threads)

C) Many-to-many (M:N)

- multe user threads mapate pe mai multe kernel threads
- incearca sa combine avantajele
- e mai complex de implementat

Asociere de tinut minte:

- M:1 = rapid, dar blocaj + fara multicore
- 1:1 = simplu + multicore, dar cost
- M:N = “best of both worlds”, dar complex

User thread = “fire gestionate de program”, OS nu le vede.

OS-ul:

- NU stie ca thread-ul exista
- vede **un singur proces / kernel thread**

Kernel thread = Thread gestionat direct de sistemul de operare.

Exemplu

User threads (M:1)

Ai 4 user thread-uri pe 1 kernel thread:

- unul face read() blocant
- kernelul blocheaza **kernel thread-ul** => **toate user thread-urile se opresc**

Kernel threads (1:1)

Ai 4 kernel thread-uri:

- unul face read() blocant
- celelalte **continua sa ruleze** pe alte core-uri

Rezumat:

- user thread: user space, M:1, rapid, fara multicore
- kernel thread: OS, 1:1, paralelism real
- syscall blocant: user → blocheaza tot; kernel → blocheaza doar unul

ASOCIERE FOARTE IMPORTANTA (EXAMEN)

System call Asteapta ce?

read date

write buffer liber

accept client

connect conexiune

wait	copil
sleep	timp
pause	semnal
sem_wait	resursa
pthread_join	thread

PROCESE vs THREAD-uri

1. Memorie

- Proces: spatiu de adrese separat
- Thread: spatiu de adrese comun (in acelasi proces)

2. Comunicare

- Proces: IPC (pipe, shm, semnale) -> mai greu
- Thread: memorie comuna -> mai usor, dar periculos

3. Cost

- Proces: creare + switch mai costisitor
- Thread: creare + switch mai ieftin (de obicei)

4. Izolare / siguranta

- Proces: crash izolat (de regula)
- Thread: crash intr-un thread poate termina intreg procesul

5. Sincronizare

- Proces: mai putine race condition pe memorie (nu e partajata implicit)
- Thread: multe race condition -> ai nevoie de mutex/semafor

1.4.2 Cand aleg proces vs cand aleg thread (cu exemple)

Aleg PROCES cand:

- vreau **izolare** si siguranta
- rulez componente independente
- am nevoie de permisiuni/drepturi diferite

Exemplu: browser: tab-uri izolate in procese separate (daca unul crapa, nu crapa tot)

Aleg THREAD cand:

- vreau sa impart aceeași memorie și sa comunic rapid
- am sarcini care se pot paraleliza
- am server cu multe cereri

Exemplu: server web: un thread per client / pool de thread-uri

Ce este un Thread Pool

Thread pool = un set fix de thread-uri create dinainte, care asteapta task-uri și le executa pe rand.

- thread-urile NU se creeaza pentru fiecare cerere
 - ele sunt **reutilizate**
 - task-urile sunt puse într-o **coada**
-

Cum functioneaza

1. La pornirea aplicatiei:

- se creeaza **N thread-uri**
- ele intra in stare de asteptare

2. Cand apare un task:

- task-ul este pus într-o **coada**
- un thread liber îl preia

3. Thread-ul:

- executa task-ul
- revine in pool
- asteapta urmatorul task

Exemplu

// pseudo-cod

```
create_pool(4);
```

```
while (true) {
```

```
task = accept_request();  
  
enqueue(task);  
  
}
```

CE SUNT SEMNALELE (Signals)

Un signal este un mecanism prin care kernelul notifica un proces ca a avut loc un eveniment.

Semnalul:

- NU trimite date
- e doar o **notificare** („s-a intamplat ceva!”)

Exemple de evenimente care genereaza semnale

- apesi **Ctrl+C** → SIGINT
 - copilul se termina → SIGCHLD
 - acces invalid la memorie → SIGSEGV
 - timer expirat → SIGALRM
 - kill(pid, SIGTERM) → semnal trimis explicit
-

CUM FUNCTIONEAZA SIGNAL HANDLING

1. Semnalul este generat

De:

- kernel (eroare, I/O, timer)
- alt proces (kill)
- utilizator (Ctrl+C)

2. Semnalul este livrat

- kernelul il **asociaza unui proces** (sau thread)

3. Semnalul este tratat (handled)

In unul din doua moduri:

a) Default handler

- definit de kernel
- fiecare semnal are unul

Exemple:

- SIGINT → termina procesul
- SIGSEGV → termina + core dump
- SIGCHLD → ignorat (default)

b) User-defined handler

- programatorul il defineste
- **suprascrie** comportamentul default

```
void handler(int sig) {
    printf("Am primit semnal\n");
}

signal(SIGINT, handler);
```

CE INSEAMNA DEFAULT vs USER HANDLER

Default handler

- ruleaza daca NU definesti tu altceva
- este controlat de kernel

User-defined handler

- functia ta
- kernelul o apeleaza cand vine semnalul
- NU toate semnalele pot fi suprascrise:
 - SIGKILL
 - SIGSTOP

Thread cancellation = terminarea unui thread inainte sa-si fi terminat executia.

- thread-ul care va fi oprit = **target thread**
- anularea este ceruta de **alt thread** (de obicei main)

Cum se face in POSIX (pthread)

```
pthread_cancel(tid); // cere anulara thread-ului  
pthread_join(tid, NULL); // asteapta sa se termine
```

Cum controlezi comportamentul

Activare / dezactivare anulare

```
pthread_setcancelstate(PTHREAD_CANCEL_DISABLE, NULL);  
  
// sectiune critica  
  
pthread_setcancelstate(PTHREAD_CANCEL_ENABLE, NULL);
```

Tipul de anulare

```
pthread_setcanceltype(PTHREAD_CANCEL_DEFERRED, NULL); // default  
  
pthread_setcanceltype(PTHREAD_CANCEL_ASYNCHRONOUS, NULL);
```

Concluzie:

- pthread_cancel → cere anulare
- async = imediat, periculos
- deferred = sigur, default
- cancellation points = read, sleep, wait
- pthread_join → curata thread-ul

SINCRONIZARE SI CONCURENTA

Race condition si sectiune critica

Race condition = rezultatul depinde de ordinea/intercalarea executiei thread-urilor/proceselor care acceseaza date comune.

Asociere: “cine ajunge primul strica rezultatul”.

Exemplu clasic (incrementare contor)

Ai doua thread-uri, ambele fac cnt++.

cnt++ NU e o singura instructiune, ci:

1. citește cnt din memorie
2. adună 1
3. scrie înapoi

Dacă intercalarea e așa:

- cnt = 5
- T1 citește 5
- T2 citește 5
- T1 scrie 6
- T2 scrie 6

Rezultat final: 6 (dar trebuia 7) - Asta e race condition.

Secțiune critică = bucată de cod care accesează o resursă partajată (variabilă, structură, fișier, etc.) și trebuie executată atomic (fără intercalare).

Ex:

- modificare cnt
- inserare în listă
- update într-un buffer comun

Condițiile pentru o soluție corectă (pt secțiune critică)

1) Mutual exclusion

- **Ce înseamnă:** maxim **1 thread** intră în secțiunea critică (SC) la un moment dat.
De ce: altfel ai race condition.
- **Asociere:** 1 cheie la baie.

2) Progress

- **Ce înseamnă:** dacă SC e **liberă** și există thread-uri care vor să intre, sistemul trebuie să permită ca **unul să intre** (nu sta blocat “degeaba”).
De ce: să nu existe situații în care e liber dar nimeni nu intră.
- **Asociere:** usa e liberă → cineva intră, nu aștepta fără motiv.

3) Bounded waiting

- **Ce inseamna:** daca un thread cere sa intre, nu poate fi sarit la infinit → exista o limita, deci **nu apare starvation**.
De ce: sa nu “moara de foame” un thread.
- **Asociere:** coada corecta (nu te baga lumea in fata la infinit).

Mutex

Mutex = lock binar care permite accesul unui singur thread la sectiunea critica.

Operatii:

- lock(m) / pthread_mutex_lock
- unlock(m) / pthread_mutex_unlock

Asociere: cheia de la baie: doar unul o are.

Ce se intampla daca mutex e ocupat

Depinde de implementare:

- **blocking mutex:** thread-ul e pus in WAITING (nu consuma CPU)
- **spinlock:** thread-ul “se invarte” si verifica repetat (consuma CPU)

Pentru examen, e ok sa spui:

Daca mutex e ocupat, thread-ul asteapta pana devine liber.

Exemplu(fix race condition)

```
pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;
```

```
int cnt = 0;
```

```
void* worker(void* arg){
    for(int i=0;i<100000;i++){
        pthread_mutex_lock(&m);
        cnt++;
        pthread_mutex_unlock(&m);
    }
}
```



```
return NULL;  
}
```

Erori clasice cu mutex (capcane de examen)

1. Uiti unlock

- toate thread-urile se blocheaza -> deadlock

2. Lock-uri in ordine diferita (deadlock clasic)

- T1: lock(A) apoi lock(B)
- T2: lock(B) apoi lock(A)
=> blocaj circular

3. Mutex in jurul prea multui cod

- scade paralelismul
 - performanta slaba
-

Semafor

Semafor = contor sincronizat folosit pentru a controla accesul la resurse.

Operatii:

- **wait(P)**: daca semafor > 0 , scade si intra; altfel blocheaza
 - **signal(V)**: creste si trezeste eventual un thread
-

Tipuri

Semafor binar

- valori 0/1
- se comporta ca un mutex (aproape)

Semafor contor (counting)

- poate avea valori 0..N
- modeleaza "N resurse identice"

Asociere:

- binar = usa cu o cheie
 - contor = parcare cu N locuri
-

Exemplu: N resurse (clasic)

Ai 3 imprimante, maxim 3 thread-uri pot printa simultan:

- sem = 3
 - fiecare print:
 - wait(sem)
 - print
 - signal(sem)
-

Condition variables

Condition variable = mecanism ca un thread sa astepte o conditie logica (ex: buffer not empty) fara busy-waiting.

Se folosesc impreuna cu mutex.

Operatii:

- wait(cond, mutex)
 - signal(cond) (trezeste unul)
 - broadcast(cond) (trezeste toti)
-

Conditia se verifica in while, NU in if:

```
pthread_mutex_lock(&m);  
while (!conditie) {  
    pthread_cond_wait(&cv, &m);  
}  
... folosesti resursa ...  
pthread_mutex_unlock(&m);
```

De ce while?

- pot exista **spurious wakeups**
- sau se trezeste, dar alt thread consuma resursa inaintea lui

Busy waiting (spin)

Thread-ul consuma CPU intr-un loop asteptand lock-ul (ex: spinlock). Bun doar daca CS e foarte scurta.

Monitor

Mecanism de nivel inalt: un modul cu variabile private + proceduri; garanteaza ca doar un thread e activ in monitor la un moment dat. Are condition variables.

Deadlock

= situatia in care un set de procese/fire se blocheaza reciproc pentru ca:

- fiecare tine cel putin o resursa
- fiecare asteapta o resursa tinuta de altul
- si nimeni nu mai poate progresa.

Modelul clasic: **request** → **use** → **release** pentru resurse.

Cele 4 conditii necesare (toate trebuie sa fie adevarate simultan)

1. **Mutual exclusion (excluziune mutuala)**
O resursa poate fi folosita de **un singur proces** la un moment dat (ex: imprimanta, mutex).
2. **Hold and wait (tine si asteapta)**
Procesul **tine deja** o resursa si **cere inca una**.
3. **No preemption (fara preemptie)**
Resursa nu poate fi luata “cu forta”; se elibereaza doar voluntar de procesul care o tine.
4. **Circular wait (asteptare circulara)**
Exista un ciclu de procese $T_0 \rightarrow T_1 \rightarrow \dots \rightarrow T_0$, fiecare asteptand o resursa de la urmatorul.

Exemplu rapid (semnaleaza deadlock clar)

Ai doua lock-uri (mutexuri) S1 si S2, initial libere.

- T1: wait(S1) apoi wait(S2)
- T2: wait(S2) apoi wait(S1)

Daca T1 apuca S1 si T2 apuca S2, fiecare asteapta al doilea -> deadlock.
(Exact genul asta apare si la “dining philosophers” ca idee: toti iau cate un betisor si asteapta al doilea.)

Starvation

Un thread asteapta “pe veci” pentru ca altii sunt serviti mereu inainte (politica nedreapta).

Priority inversion

Un thread low-priority tine un lock necesar unui high-priority; high asteapta, iar medium ruleaza si agraveaza. Solutie: **priority inheritance**.

Memory reordering + memory barrier

CPU/compiler poate reordona operatii -> algoritmi software (ex Peterson) pot pica fara bariere. Memory barrier impune ordinea/visibilitatea operatiilor intre thread-uri.

1) Mutual exclusion cu mutex

```
lock(m);  
  
    // critical section  
  
unlock(m);
```

Explicatie: lock garanteaza ca un singur thread intra; unlock elibereaza.

2) Mutual exclusion cu semafor binar

```
// init mutex = 1  
  
wait(mutex);  
  
    // critical section  
  
signal(mutex);
```

3) N resurse identice (semafor contor)

Ex: ai 3 imprimante -> max 3 procese pot printa simultan.

```
// init S = 3
```

```
wait(S);    // ia o resursa

    use_resource();

signal(S); // elibereaza
```

4) Ordonare: S1 trebuie inainte de S2

```
// init sync = 0

// Thread A

S1();

signal(sync);

// Thread B

wait(sync);

S2();
```

5) Producer–Consumer (bounded buffer) cu semafoare

Buffer de dimensiune N.

```
// init

semaphore mutex = 1;

semaphore empty = N; // cate locuri libere

semaphore full  = 0; // cate ocupate

// Producer

while(true){

    item x = produce();

    wait(empty);

    wait(mutex);

    put(x);

    signal(mutex);
```

```

    signal(full);
}

// Consumer
while(true){
    wait(full);
    wait(mutex);
    item x = get();
    signal(mutex);
    signal(empty);
    consume(x);
}

```

Explicatie:

- `empty/full` tin cont de spatiu/elemente
- `mutex` protejeaza accesul la buffer

6) Readers–Writers (varianta clasica, fara starvare garantata)

```

semaphore mutex = 1;    // protejeaza readcount
semaphore wrt  = 1;    // lock pentru scriitori
int readcount = 0;

Reader(){
    wait(mutex);

    readcount++;

    if(readcount==1) wait(wrt); // primul reader blocheaza scriitorii

    signal(mutex);
}

```

```

    read_data();

    wait(mutex);

    readcount--;

    if(readcount==0) signal(wrt); // ultimul reader elibereaza

    signal(mutex);
}

Writer(){

    wait(wrt);

    write_data();

    signal(wrt);
}

```

7) Monitor + condition variable (pattern corect)

```

monitor M {

    condition cv;

    bool ready = false;

    void A(){

        // produce something

        ready = true;

        cv.signal();

    }

    void B(){

        while(!ready) cv.wait();

        // use produced thing

    }

}

```

Explicatie: `wait()` elibereaza monitorul si doarme; `signal()` trezeste.

8) Deadlock cu doua lock-uri (capcana clasica)

```
// P0

wait(S);

wait(Q);

// P1

wait(Q);

wait(S);
```

Fix: impui aceeasi ordine peste tot (ex: mereu S apoi Q).

9) CAS / atomic increment (daca se cere)

```
int atomic_inc(int* v){

    int old;

    do{

        old = *v;

    }while(!CAS(v, old, old+1));

    return old+1;

}
```

1. Functia `compare_and_swap` (ATOMICA – hardware)

```
int compare_and_swap(int *value, int expected, int new_value)
{
    int temp = *value;          // citeste valoarea veche
    if (*value == expected)     // compara cu expected
        *value = new_value;    // seteaza noua valoare
    return temp;                // returneaza valoarea veche
}
```



```
}
```

- Aceasta functie este **atomica** (implementata de CPU).

2. Variabila partajata (lock)

```
int lock = 0;    // 0 = liber, 1 = ocupat
```

3. Codul unui thread / proces (spinlock cu CAS)

```
while (true) {  
    while (compare_and_swap(&lock, 0, 1) != 0)  
        ;    // asteptare activa (spin)  
    /* CRITICAL SECTION */  
    // aici intra un singur thread  
    lock = 0;    // elibereaza lock-ul  
    /* REMAINDER SECTION */  
}
```

Intrarea in sectiunea critica

1. Thread-ul apeleaza: `compare_and_swap(&lock, 0, 1)`
 2. Daca `lock == 0`:
 - CAS returneaza 0
 - lock devine 1
 - thread-ul **intra in critical section**
 3. Daca `lock == 1`:
 - CAS returneaza 1
 - lock NU se schimba
 - thread-ul ramane in **while** (asteapta)
-

Iesirea din sectiunea critica

```
lock = 0;
```

- lock devine liber
 - alt thread poate intra
-

Rezolva CAS problema sectiunii critice?

DA:

- mutual exclusion (doar un thread intra)
- progress (cineva intra)
- functioneaza pe multi-core

NU:

- NU garanteaza bounded waiting
- poate aparea **starvation**

exact ca TAS, este un **spinlock simplu**.

De ce CAS simplu NU are bounded waiting

Pentru ca:

- thread-urile concureaza agresiv
- acelasi thread poate castiga de mai multe ori
- nu exista ordine / rand

un thread poate astepta **la infinit**

CAS cu bounded waiting

Variabile partajate

```
int lock = 0;
```

```
boolean waiting[n];
```

Cod pentru procesul i

```
while (true) {  
    waiting[i] = true;  
    int key = 1;  
    while (waiting[i] && key == 1)
```

```

        key = compare_and_swap(&lock, 0, 1);
waiting[i] = false;
/* CRITICAL SECTION */
int j = (i + 1) % n;
while ((j != i) && !waiting[j])
    j = (j + 1) % n;
if (j == i)
    lock = 0;
else
    waiting[j] = false;
/* REMAINDER SECTION */
}

```

Cum functioneaza

- `waiting[i]` = „procesul i asteapta randul”
 - CAS asigura **exclusivitatea**
 - vectorul `waiting[ ]` asigura **ordine si limita de asteptare**
 - cand un proces iese:
 - fie elibereaza lock-ul
 - fie „da stafeta” urmatorului proces care asteapta
 - **nu exista starvation**
 - fiecare proces intra dupa un numar finit de pasi
-

10) TAS

1. Functia test_and_set (atomica)

```

boolean test_and_set(boolean *target)
{
    boolean rv = *target;  // citeste valoarea veche a lock-ului
    *target = true;        // seteaza lock-ul pe true (ocupat)
}

```

```
        return rv;                // returneaza valoarea veche
    }
```

Aceasta functie trebuie sa fie **atomica** (hardware).

2. Variabila partajata (lock)

```
boolean lock = false; // false = liber, true = ocupat
```

3. Codul unui thread / proces

```
do {
    while (test_and_set(&lock))
        ; // asteptare activa (spin)

    /* CRITICAL SECTION */

    // aici intra DOAR un singur thread

    lock = false; // elibereaza lock-ul

    /* REMAINDER SECTION */

    // cod care nu necesita sincronizare
} while (true);
```

Cum functioneaza

1. Thread-ul apeleaza `test_and_set(&lock)`
2. Daca lock era `false`:
 - TAS returneaza `false`
 - lock devine `true`
 - thread-ul intra in **critical section**

3. Daca lock era `true`:
 - TAS returneaza `true`
 - thread-ul ramane in `while` (asteapta)
4. La final:
 - thread-ul face `lock = false`
 - alt thread poate intra

Resource Allocation Graph (RAG)

Este un graf cu 2 tipuri de noduri:

- **procese/fire**: $T_1, T_2, \dots$
- **tipuri de resurse**: $R_1, R_2, \dots$ (fiecare cu 1 sau mai multe instante).

Muchii (foarte important)

- **request edge**: $T_i \rightarrow R_j$ (T_i CERE un exemplar din R_j)
- **assignment edge**: $R_j \rightarrow T_i$ (un exemplar din R_j este ALOCAT lui T_i)

Regula de deadlock cu cicluri

- **Daca fiecare resursa are 1 singura instanta:**
ciclu in graf = deadlock sigur.
- **Daca exista resurse cu mai multe instante:**
ciclu = posibil deadlock, nu garantat (poate exista o instanta libera care sparge blocajul).

Mini-exemplu (1 instanta per resursa)

T_1 tine R_1 si cere R_2

T_2 tine R_2 si cere R_1

Graf: $R_1 \rightarrow T_1, T_1 \rightarrow R_2, R_2 \rightarrow T_2, T_2 \rightarrow R_1$ (ciclu) $\Rightarrow$ deadlock sigur.

Prevenire (deadlock prevention)

Prevention = “te asiguri ca sistemul nu intra niciodata in deadlock” prin invalidarea uneia din cele 4 conditii.

A) Rupe hold-and-wait

Regula: procesul **cere toate resursele de la inceput** (sau cere resurse doar cand nu tine nimic).

- Pro: elimina deadlock.
 - Contra: utilizare slaba a resurselor, poate duce la starvation (proces care asteapta mult).
- ch8

B) Rupe no-preemption

Regula: daca un proces tine resurse si cere una indisponibila, **i se iau (elibereaza) resursele tinute** si incearca iar mai tarziu.

- Pro: elimina deadlock.
- Contra: greu de aplicat pentru resurse “nedivizibile” (ex: imprimanta in mijlocul tiparii).

C) Rupe circular-wait (cea mai folosita)

Impunem o **ordonare totala** a resurselor (de ex. $R1 < R2 < R3 < \dots$) si cerem regula:

fiecare proces are voie sa ceara resurse doar in ordine crescatoare.

Ex: ai doua mutexuri $M1=1$ si $M2=5$. Nu ai voie sa iei $M2$ si apoi $M1$.

De tinut minte: asta e “standard-ul” pe mutexuri/lock-uri in sisteme reale.

Evitare (deadlock avoidance)

Avoidance = sistemul **decide dinamic** daca acorda o cerere astfel incat sa ramana intr-o stare “sigura”.

Safe state vs Unsafe state

- **Safe state:** exista o ordine in care procesele pot termina unul cate unul, folosind resursele disponibile + resursele eliberate de cele care termina.
- ch8
- **Unsafe state:** nu e deadlock neaparat, dar **exista posibilitatea** sa ajungi in deadlock.
- ch8

Regula: avoidance nu lasa sistemul sa intre in **unsafe**.

Banker’s Algorithm (pentru mai multe instante / mai multe resurse)

Structuri (le inveti exact asa)

Sa fie:

- $n = \text{nr procese } (T_0..T_{n-1})$
- $m = \text{nr tipuri de resurse } (A,B,C,...)$

Avem:

- **Available[m]**: cate instante libere din fiecare resursa
- **Max[n][m]**: maximul declarat ca poate cere procesul
- **Allocation[n][m]**: cate are alocat acum
- **Need[n][m] = Max - Allocation**

Safety check (algoritmul de siguranta) – PASI DE EXAMEN

1. $Work = Available$
 2. $Finish[i] = false$ pentru toate procesele
 3. Cauta un i cu:
 - $Finish[i] = false$
 - $Need[i] \leq Work$ (component-wise)
 4. Daca gasesti:
 - $Work = Work + Allocation[i]$
 - $Finish[i] = true$
 - repeti pasul 3
 5. Daca la final toate $Finish$ sunt true => **safe** (exista safe sequence).
- ch8

Resource-request algorithm (cand T_i cere $Request[i]$)

1. Verifica $Request[i] \leq Need[i]$ (altfel e cerere invalida)
 2. Verifica $Request[i] \leq Available$ (daca nu, asteapta)
 3. “Prefa-te ca aloci”:
 - $Available -= Request$
 - $Allocation[i] += Request$
 - $Need[i] -= Request$
 4. Ruleaza safety check:
 - daca safe => accepti cererea
 - daca unsafe => revii la starea initiala si procesul asteapta
- ch8

Mini-exercitiu (stil examen, sa exersezi mecanic)

Resurse A,B:

- $Available = (1,1)$
Procese:
- $T_0: Max(1,1), Allocation(1,0) \Rightarrow Need(0,1)$
- $T_1: Max(1,1), Allocation(0,1) \Rightarrow Need(1,0)$

Intrebare: sistem safe?

- $Work=(1,1)$
- $T0: Need(0,1) \leq Work \Rightarrow da \Rightarrow Work=Work+(1,0)=(2,1)$
- $T1: Need(1,0) \leq Work \Rightarrow da \Rightarrow Work=Work+(0,1)=(2,2)$
 $\Rightarrow$ safe, sequence $\langle T0, T1 \rangle$.

CPU SCHEDULING (planificarea procesorului)

Arrival time (AT) / Start

Momentul cand procesul intra in coada Ready (cand “apare”).

Ex: P1 are $AT=2 \Rightarrow$ pana la $t=2$ nu exista in coada.

Burst time (BT) / CPU

Timpul total de CPU de care are nevoie procesul (cat trebuie sa ruleze efectiv pe CPU).

Ex: $BT=5 \Rightarrow$ procesul are nevoie de 5 unitati de timp pe CPU.

Completion time (CT)

Momentul cand procesul termina complet executia (cand iese din sistem).

Turnaround time (TAT)

Cat timp a stat procesul in sistem (de la sosire pana la terminare).

Formula:

- $TAT = CT - AT$

Waiting time (WT)

Cat timp a asteptat in coada Ready (nu include timpul cat a rulat pe CPU).

Formula standard (cea mai folosita la examen):

- $WT = TAT - BT$

Response time (RT) (daca cere)

Cat timp asteapta pana ruleaza prima data pe CPU.

- $RT = (first_start_time) - AT$

Exemplu

P1: AT=0, BT=4

Daca termina la CT=7:

- $TAT = 7 - 0 = 7$
- $WT = 7 - 4 = 3$

Daca a inceput prima data la $t=2$:

- $RT = 2 - 0 = 2$
-

Preemptiv vs Nepreemptiv

- **nepreemptiv**: odata ce procesul intra pe CPU, ruleaza pana termina sau se blocheaza
- **preemptiv**: OS poate lua CPU de la un proces (ex: expira quantum, apare proces mai scurt, prioritate mai mare)

Exemple:

- FCFS = nepreemptiv
- SJF (clasic) = nepreemptiv
- SRTF = preemptiv
- Round Robin = preemptiv

Scheduling Criteria

- CPU utilization (max)
 - Throughput (max)
 - Turnaround time (min)
 - Waiting time (min)
 - Response time (min)
-

Algoritmi obligatorii

A) FCFS (First Come First Served)

- Primul venit, primul servit.

PASI

1. Ordonezi procesele dupa AT (arrival)
2. CPU ia primul disponibil
3. Daca nu e nimeni disponibil, timpul sare la urmatorul AT
4. Faci Gantt: fiecare proces ruleaza **BT complet** (nepreemptiv)
5. Calculezi CT, apoi TAT si WT

Ex:

Procese:

- P1: AT=0, BT=4
- P2: AT=1, BT=3
- P3: AT=2, BT=2

Gantt FCFS:

- t=0: P1 ruleaza 0-4
- apoi P2 4-7
- apoi P3 7-9

CT:

- P1 CT=4
- P2 CT=7
- P3 CT=9

TAT = CT-AT:

- P1: 4-0=4
- P2: 7-1=6
- P3: 9-2=7

WT = TAT-BT:

- P1: 4-4=0
- P2: 6-3=3
- P3: 7-2=5

WT mediu = $(0+3+5)/3 = 8/3 = 2.666...$

Avantaje / Dezavantaje (2-3 randuri pentru teorie)

- simplu, fara overhead
- poate produce "convoy effect" (un proces lung tine in spate multe scurte)
- waiting time mare in medie

B) SJF (Shortest Job First) nepreemptiv -

Cand CPU devine liber, alegi procesul disponibil cu BT minim.

PASI

1. Pornesti la t=0

2. In fiecare moment cand CPU devine liber:
 - bagi in “available set” toate procesele cu $AT \leq t$
 - alegi procesul cu BT minim
3. Ruleaza pana termina (nepreemptiv)
4. Calculezi CT, TAT, WT

Exemplu rezolvat (aceleasi procese)

P1: $AT=0$ $BT=4$

P2: $AT=1$ $BT=3$

P3: $AT=2$ $BT=2$

La $t=0$: disponibil doar P1 => ruleaza 0-4

La $t=4$: disponibile P2, P3 => alegi BT minim => P3 (2) ruleaza 4-6

Apoi P2 6-9

CT:

- P1=4
- P3=6
- P2=9

TAT:

- P1: $4-0=4$
- P3: $6-2=4$
- P2: $9-1=8$

WT:

- P1: $4-4=0$
- P3: $4-2=2$
- P2: $8-3=5$

Observatie: SJF scade waiting time fata de FCFS.

Avantaje / Dezavantaje

- minimizeaza waiting time mediu (teoretic)
- necesita sa stii BT dinainte (estimare)
- poate produce starvation pentru procese lungi

C) SRTF (Shortest Remaining Time First) = SJF preemptiv

Mereu ruleaza procesul cu **timp ramas minim**. Daca apare unul mai scurt, il intrerupe.

PASI

1. Tii pentru fiecare proces “remaining time”
2. In timp:
 - la orice eveniment (sosire proces nou sau terminare):
 - alegi procesul cu remaining minim dintre cele sosite
3. Daca apare un proces nou cu remaining mai mic => preemptie
4. Continui pana toate termina
5. Calculezi CT, apoi TAT/WT

Ex: (clasica diferenta fata de SJF)

Procese:

- P1: AT=0 BT=8
- P2: AT=1 BT=4
- P3: AT=2 BT=2

Timeline:

- t=0-1: ruleaza P1 (ramane 7)
- la t=1 apare P2 (4) < 7 => preemptie
- t=1-2: ruleaza P2 (ramane 3)
- la t=2 apare P3 (2) < 3 => preemptie
- t=2-4: ruleaza P3 (termina la 4)
- acum P2 ramane 3, P1 ramane 7 => ruleaza P2
- t=4-7: ruleaza P2 (termina la 7)
- t=7-14: ruleaza P1 (termina la 14)

CT:

- P3 CT=4
- P2 CT=7
- P1 CT=14

TAT:

- P1: 14-0=14
- P2: 7-1=6
- P3: 4-2=2

WT = TAT - BT:

- P1: 14-8=6
- P2: 6-4=2
- P3: 2-2=0

D) Round Robin (RR)

Fiecare proces primește CPU pe un interval fix numit **quantum q**. Dacă nu termină în q, se pune la coada la final.

PASI

1. Sortezi procesele după AT
2. Tii o **coada Ready FIFO**
3. Când timpul curent t avansează:
 - bagi în coada toate procesele cu $AT \leq t$ care au sosit între timp
4. Scoti primul din coada:
 - rulează **min(q, remaining)**
5. Dacă mai are remaining > 0:
 - îl pui la finalul cozii
6. Repeta până termină toate
7. Calculezi CT, apoi WT/TAT

Regula importantă de examen:

Când un proces rulează între t și t+q, orice proces nou sosit în interval **întra în coada** și va fi luat după ce se termină slice-ul curent (nu intrerupe imediat).

Ex:

Procese:

- P1: AT=0 BT=5
 - P2: AT=1 BT=3
 - P3: AT=2 BT=4
- Quantum q=2

Pas cu pas:

t=0: coada = [P1]

- rulează P1 0-2 (ramane 3)
între timp sosesc P2 (t=1), P3 (t=2) => coada devine [P2, P3]
P1 nu termină => îl pui la final => coada [P2, P3, P1]

t=2:

- rulează P2 2-4 (ramane 1) => coada [P3, P1, P2]

t=4:

- rulează P3 4-6 (ramane 2) => coada [P1, P2, P3]

t=6:

- ruleaza P1 6-8 (ramane 1) => coada [P2, P3, P1]

t=8:

- ruleaza P2 8-9 (termina) => coada [P3, P1]

t=9:

- ruleaza P3 9-11 (termina) => coada [P1]

t=11:

- ruleaza P1 11-12 (termina)

CT:

- P2 CT=9
- P3 CT=11
- P1 CT=12

TAT:

- P1: 12-0=12
- P2: 9-1=8
- P3: 11-2=9

WT = TAT - BT:

- P1: 12-5=7
- P2: 8-3=5
- P3: 9-4=5

E) Priority Scheduling (daca apare)

Ideea: alegei procesul cu prioritatea cea mai mare (sau cea mai mica numeric, depinde de conventie).

- nepreemptiv: ruleaza pana termina
- preemptiv: daca apare unul cu prioritate mai mare, il preia

Problema: **starvation** (un proces cu prioritate mica poate astepta enorm)

Solutie: **aging** (cresti prioritatea in timp)

Starvation apare atunci cand un proces asteapta foarte mult (sau chiar infinit) si NU primește CPU, desi sistemul functioneaza normal.

Procesul NU este blocat (nu asteapta I/O).

Doar ca **altii sunt mereu preferati.**

Cand apare starvation

1. In Priority Scheduling

Daca:

- procesele cu prioritate mare tot apar
- scheduler-ul alege mereu prioritatea cea mai mare

Atunci:

- procesele cu prioritate mica **nu mai ruleaza niciodata**

Exemplu clar

Procese:

- P1: prioritate 1 (mare)
- P2: prioritate 10 (mica)

Daca tot apar procese ca P1:

- CPU ruleaza mereu P1
- P2 asteapta la nesfarsit

-> **P2 sufera starvation**

2. In SJF / SRTF

SJF alege procesul cu **job cel mai scurt.**

Daca:

- tot apar procese cu burst mic
- exista un proces cu burst mare

Atunci:

- procesul mare este mereu amanat

Exemplu

- P1: BT = 100
- P2, P3, P4...: BT = 1

P1 este tot amanat → **starvation**

3. In sisteme cu mai multe cozi (Multilevel queues)

- cozile de sus au prioritate mai mare
- daca sunt mereu procese in cozile de sus
- cele de jos nu mai ajung la CPU

-> starvation in cozile inferioare

Cand NU apare starvation

- **FCFS** (toata lumea asteapta la rand)
- **Round Robin** (fiecare primeste CPU periodic)

-> De aceea RR este considerat **fair**.

Aging este o tehnica de prevenire a starvation-ului.

Ideea: cu cat un proces asteapta mai mult, cu atat ii creste prioritatea.

Cand

Cand folosesti algoritmi cu prioritate

Sau orice algoritm unde exista risc de starvation

Cum functioneaza aging

1. Un proces intra in coada cu prioritate mica
2. Sta in asteptare
3. Dupa un anumit timp:

- OS ii **imbunatateste prioritatea**
- 4. Dupa suficient timp:
- procesul ajunge sa fie ales

=> **garanteaza ca va rula**

Exemplu de aging

Fara aging:

- P2 (prioritate mica) asteapta infinit

Cu aging:

- la fiecare 10 unitati de timp:
 - prioritatea lui P2 creste
- dupa un timp:
 - P2 ajunge cu prioritate suficient de mare
 - este ales

1) Multilevel Queue (MLQ) – “mai multe cozi FIXE”

Procesele sunt impartite in **categorii** (clase) si fiecare categorie are **coada ei**. Un proces este pus intr-o coada si **ramane acolo**.

Exemple de categorii:

- system processes
- interactive (foreground)
- batch
- background

Cum functioneaza

1. Cand un proces apare, OS decide **ce tip este** (ex: interactive vs batch).
2. Il pune in coada corespunzatoare (Q0, Q1, Q2...).
3. Scheduler-ul alege mai intai **din coada cu prioritate mai mare**.
4. In interiorul fiecarei cozi se aplica un algoritm (ex: RR in Q0, FCFS in Q1).

Exemplu clasic

- Q0 (interactive): Round Robin, quantum mic
- Q1 (batch): FCFS

Regula intre cozi:

- mereu alegi din Q0 daca nu e goala
- daca Q0 e goala, alegi din Q1

Ce e important

- cozi **statice**
- un proces **nu se muta** intre cozi
- risc mare de starvation pentru cozile de jos daca Q0 e mereu plina

Unde apare starvation si cum se rezolva aici

- apare daca procesele din coada de sus vin continuu
 - se poate rezolva cu:
 - “time slice” intre cozi (ex: 80% CPU pentru Q0, 20% pentru Q1)
 - sau aging (rar in MLQ, mai des in MLFQ)
-

2) Multilevel Feedback Queue (MLFQ) – “cozi + feedback + mutare”

Ai tot mai multe cozi, dar:

procese **pot fi mutate** intre cozi in functie de cum se comporta.

“Feedback” = OS se uita daca procesul:

- foloseste mult CPU (CPU-bound)
- sau se opreste des pentru I/O (interactive)

Scop:

- interactive sa para rapid (raspuns bun)
- batch sa ruleze in fundal fara sa moara de foame

Cum functioneaza

Ai cozi Q0 (cea mai inalta) ... Qk (cea mai joasa)

Reguli tipice:

1. Un proces nou intra **in coada de sus (Q0)**.
2. In Q0 ruleaza cu **Round Robin** si un quantum mic.
3. Daca procesul NU termina in quantum:
 - este “penalizat” si coboara in coada urmatoare (Q1).

4. In Q1 are quantum mai mare (sau tot RR).
5. Daca iar nu termina, coboara in Q2, etc.
6. Ultima coada (Qk) este de obicei FCFS.

Prioritate:

- scheduler-ul alege mereu din coada cea mai de sus care nu e goala.

De ce e “feedback”

Pentru ca OS “invata”:

- daca procesul tot consuma CPU si nu termina repede => il coboara
- daca procesul e interactive (face I/O des) => ramane sus sau urca

Ce se intampla cu procesele interactive

De obicei:

- ele ruleaza putin, apoi fac I/O (se blocheaza)
- cand revin, sunt tratate bine (raman in cozi de sus)
Rezultat: raspuns rapid la tastatura/mouse.

Cum previne starvation

MLFQ foloseste de obicei “boosting”:

- din cand in cand, OS muta toate procesele inapoi in Q0
(sau creste prioritatea celor care asteapta mult)
Asta este o forma de aging.

Exponential Averaging

Ce este:

Metoda folosita de OS pentru a **estima urmatorul CPU burst** al unui proces (folosita in **SJF / SRTF**).

De ce:

OS nu stie viitorul → estimeaza pe baza trecutului.

Formula: $\tau(n+1) = \alpha \cdot t(n) + (1 - \alpha) \cdot \tau(n)$

Unde:

- $\tau(n+1)$ = estimarea urmatorului burst
 - $t(n)$ = ultimul CPU burst real
 - $\tau(n)$ = estimarea veche
 - $\alpha \in [0,1]$
-

Rolul lui α

- $\alpha = 0 \rightarrow$ ignori ultimul burst (conteaza doar trecutul vechi)
 - $\alpha = 1 \rightarrow$ conteaza doar ultimul burst
 - $0 < \alpha < 1 \rightarrow$ compromis (caz real)
-

Idee importanta

Burst-urile **recente conteaza mai mult**, cele vechi **din ce in ce mai putin** (ponderi care scad exponential).

Date:

$$\alpha = 0.5$$

$$\tau(0) = 10$$

$$t(1) = 6$$

Calcul:

$$\tau(1) = 0.5 \cdot 6 + 0.5 \cdot 10 = 3 + 5 = 8$$

Estimarea urmatorului CPU burst = **8**

THREAD SCHEDULING : CPU-ul planifica THREAD-uri, nu procese.

Tipuri de thread-uri

- **User-level threads** $\rightarrow$ gestionate de librerie
- **Kernel-level threads** $\rightarrow$ gestionate de OS

Process Contention Scope (PCS)

- scheduling **in interiorul procesului**
- thread-urile aceluiași proces concurează între ele
- kernel-ul vede **un singur proces**
- NU există paralelism pe mai multe core-uri

Apare la: user-level threads

System Contention Scope (SCS)

- scheduling **la nivel de sistem**
- toate thread-urile concurează pentru CPU
- kernel-ul vede fiecare thread
- există paralelism real pe multi-core

Apare la: kernel-level threads

PCS vs SCS (foarte scurt)

PCS	SCS
în proces	în sistem
bibliotecă decide	kernel decide
fără multi-core	cu multi-core
un thread blocat blochează tot	un thread blocat nu blochează restul

Avantaje / Dezavantaje

User-level threads

- context switch rapid
 - I/O blocheaza toate thread-urile
 - fara paralelism real

Kernel-level threads

- folosesc mai multe core-uri
- I/O nu blocheaza tot
 - context switch mai scump

In pthreads

- **PTHREAD_SCOPE_PROCESS** → PCS
- **PTHREAD_SCOPE_SYSTEM** → SCS
 - In practica (Linux): doar SCS

CPU Scheduler & Dispatcher

Trebuie sa stii clar:

CPU Scheduler

- alege un proces/thread din **ready queue**
- decizia se ia cand:
 - running → waiting
 - running → ready
 - waiting → ready
 - procesul termina

Dispatcher

- face efectiv schimbarea:
 - context switch
 - trecere kernel → user mode
- **dispatch latency** = timpul pierdut la schimbare

1.Multi-Processor Scheduling (conceptual)

SMP (Symmetric Multiprocessing)

- mai multe procesoare identice
- fiecare CPU poate rula orice proces/thread

- toate CPU-urile sunt egale

Ready queue

- **coada comuna**: toate CPU-urile iau din aceeași coadă
 - **cozi per-CPU**: fiecare CPU are coada lui
-

Load Balancing

Scop: toate CPU-urile să fie încărcate aproximativ egal

- **push migration**
→ OS mută procese de pe un CPU aglomerat pe unul liber
 - **pull migration**
→ un CPU liber “fura” procese de la alt CPU
-

Processor Affinity

Idee: un proces preferă să ruleze pe **același CPU** (cache)

- **soft affinity**
→ OS încearcă să-l țină pe același CPU, dar poate muta
 - **hard affinity**
→ procesul este legat strict de un CPU
-

NUMA-aware scheduling

- memorie împartită pe noduri
 - procesul rulează pe CPU-ul cel mai apropiat de memoria lui
 - reduce accesul lent la memorie
-

2.Real-Time Scheduling (doar ideea)

Soft real-time

- deadline ratat = performanță scădă
- dar sistemul continuă

Ex: video streaming

Hard real-time

- deadline ratat = **sistem esuat**
- garantii stricte de timp

Ex: airbag, control avion

Event Latency

Timpul pana un task real-time incepe sa ruleze

- **interrupt latency**
→ timp pana OS raspunde la o intrerupere
 - **dispatch latency**
→ timp pana scheduler-ul porneste task-ul
-

Algoritmi real-time

- **Rate Monotonic (RM)**
→ prioritate fixa
→ perioada mai mica = prioritate mai mare
 - **EDF (Earliest Deadline First)**
→ ruleaza task-ul cu deadline-ul cel mai apropiat
-

3.Scheduling in OS-uri reale

Linux

- **CFS (Completely Fair Scheduler)**
 - fiecare task are un **vruntime**
 - ruleaza task-ul cu **vruntime minim**
incearca sa fie “corect” cu toti
-

Windows

- scheduling bazat pe **prioritati**
 - 32 niveluri de prioritate
 - ruleaza thread-ul cu prioritate maxima
-

Solaris

- mai multe clase de scheduling
 - time-sharing = **Multilevel Feedback Queue**
-

Algorithm Evaluation

Deterministic modeling

- testezi algoritmul pe un set fix de procese
 - compari rezultate
-

Queueing models

- folosesc probabilitati
 - estimeaza waiting time / throughput
-

Little's Law: $n = \lambda \cdot W$

Unde: n = nr mediu procese in sistem , λ = rata de sosire, W = timp mediu in sistem

Ce este Rate Monotonic Scheduling (RM)

RM este un algoritm de scheduling pentru sisteme real-time (hard real-time), folosit pentru task-uri periodice.

Ideea de baza: **cu cat un task se repeta mai des (perioada mai mica), cu atat are prioritate mai mare**

Regula principala

- prioritatea este **invers proportionala cu perioada**
- **perioada mica** → **prioritate mare**
- **perioada mare** → **prioritate mica**

Rate (frecventa mare = perioada mica)

Monotonic (prioritatea e fixa, nu se schimba)

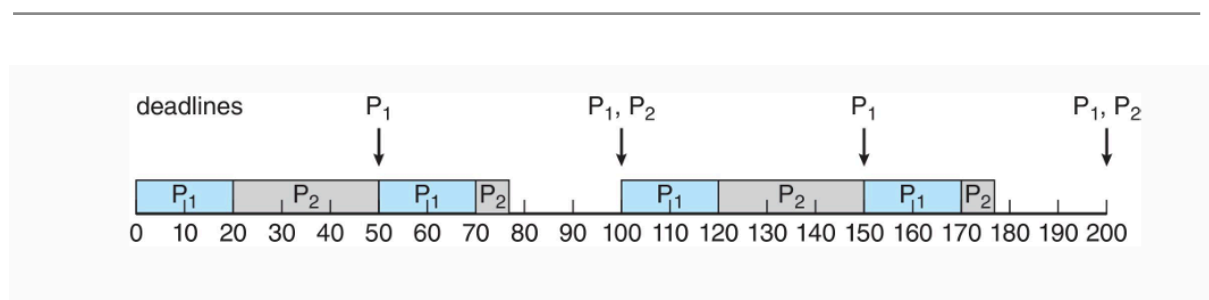
Ce inseamna „perioada”

Pentru un task periodic:

- **perioada (T)** = la cat timp apare din nou task-ul
- de obicei **deadline = perioada**

Exemplu: P1 apare la fiecare 50 ms → $T_1 = 50$, P2 apare la fiecare 100 ms → $T_2 = 100$

⇒ P1 are **prioritate mai mare** decat P2



- P1 are perioada mai mica → e albastru
- P2 are perioada mai mare → gri
- de fiecare data cand P1 apare:
 - **intrerupe** P2 (preemptie)
 - ruleaza primul

Sagetiile “deadlines” arata ca:

- P1 isi respecta deadline-ul
- P2 ruleaza doar cand P1 nu e activ

Cum functioneaza RM

1. Ai task-uri periodice
 2. Le ordonezi dupa **perioada**
 3. Atribui prioritati **fixe**: perioada mai mica → prioritate mai mare
 4. Scheduler-ul ruleaza **mereu task-ul cu prioritate mai mare**
 5. Daca apare un task cu prioritate mai mare: il **preempteaza** pe cel curent
-

Exemplu

Task-uri:

- P1: perioada = 50, CPU = 10
- P2: perioada = 100, CPU = 20

Prioritati:

- $P1 > P2$ (pentru ca $50 < 100$)

Scheduling:

- P1 ruleaza prima
- P2 ruleaza doar cand P1 nu e activ
- daca P1 apare din nou → intrerupe P2

=> dar **RM NU garanteaza mereu respectarea deadline-urilor**

Ce este EDF (Earliest Deadline First)

EDF este un algoritm de scheduling real-time care ruleaza **task-ul cu deadline-ul cel mai apropiat**.

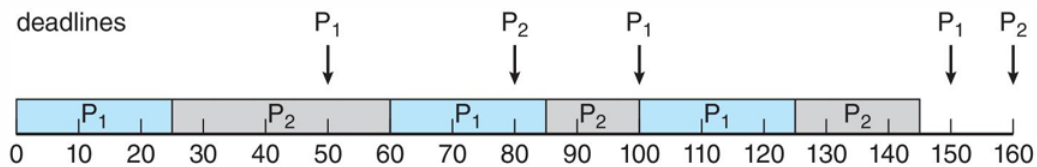
- Regula: **deadline mai apropiat = prioritate mai mare**
-

Cum functioneaza EDF

- prioritatile **NU sunt fixe**
- se **schimba dinamic**
- la fiecare decizie:
 - scheduler-ul alege task-ul cu **deadline-ul cel mai mic**

=> EDF este **preemptiv**:

- daca apare un task cu deadline mai apropiat
- il intrerupe pe cel curent



- P1 si P2 sunt task-uri periodice
- sagetile arata **deadline-urile**
- de fiecare data:
 - EDF ruleaza task-ul al carui deadline este cel mai apropiat
- **niciun deadline nu este ratat** (spre deosebire de RM)

De ce EDF este mai bun decat RM

- **RM**: prioritate fixa, dupa perioada
- **EDF**: prioritate dinamica, dupa deadline

EDF este **OPTIMAL** pe un singur CPU.

Ce inseamna “optimal”

Daca EDF nu poate programa task-urile fara sa rateze deadline-uri, **niciun alt algoritm nu poate** (pe un singur procesor).

Cand reuseste EDF

Conditie (foarte importanta): $\sum (CPU_i / Perioda_i) \leq 1$

Adica: utilizarea totala a CPU $\leq 100\%$. Daca $e \leq 1 \rightarrow$ EDF garanteaza succes.

Ce este Proportional Share Scheduling

Este un algoritm de scheduling in care **CPU-ul se imparte proportional** intre procese, in functie de **cate “shares” (actiuni/parti)** primeste fiecare.

- Ideea: Nu toti procesele sunt egale, unele primesc **o parte mai mare din CPU**.

Cum functioneaza

1. Sistemul are un numar total de **T shares**
2. Fiecare proces/aplicatie primeste **N shares** (unde $N < T$)
3. Procesul va primi:

N / T din timpul total de CPU

Garantat, indiferent ce fac celelalte procese.

Ex:

- CPU are **T = 100 shares**

Procese:

- P1: 50 shares
- P2: 30 shares
- P3: 20 shares

Atunci:

- P1 primeste 50% CPU
- P2 primeste 30% CPU
- P3 primeste 20% CPU

=> Chiar daca P1 este foarte activ, **nu poate fura mai mult**.

Exemple de algoritmi proportional share

- **Lottery Scheduling**
 - **Stride Scheduling**
 - (conceptual) Linux CFS incearca ceva asemanator (fairness)
-

Algorithm Evaluation = cum compari algoritmi de CPU scheduling ca sa vezi care e mai bun.

Nu exista “cel mai bun” in general → depinde de **ce masori** si **pe ce workload**.

Algorithm Evaluation – Deterministic Modeling

- metoda analitica de evaluare
- foloseste un **workload fix**
- compara algoritmi de scheduling
- calculeaza WT / TAT
- rezultate exacte, dar **nerealiste**

Little’s Formula

Este o **relatie matematica** care leaga:

- **cate procese sunt in medie intr-o coada**
- **cat timp asteapta**
- **cat de des sosesc**

=> Valabila **pentru ORICE algoritm de scheduling** (FCFS, RR, SJF etc.).

Formula: $n = \lambda \times W$

- **n** = numarul mediu de procese in coada
- **λ (lambda)** = rata medie de sosire (procese / unitate timp)
- **W** = timpul mediu de asteptare in coada

=> Functioneaza in **steady state** (sistem stabil: ce intra $\approx$ ce iese).

Daca:

- $\lambda = 7$ procese / secunda
- $n = 14$ procese in coada

$W = n / \lambda = 14 / 7 = 2$ secunde => un proces asteapta in medie **2 secunde**.

Evaluation by Simulation

- foloseste **trace-uri reale**
- simuleaza mai multi algoritmi
- compara statistici (WT, TAT)

- mai realist decat deterministic
- mai lent

Algoritmul Peterson (pentru 2 procese/thread-uri)

1) Problema pe care o rezolva

Ai doua thread-uri, P_0 si P_1 , care vor sa intre in **sectiunea critica (CS)** (unde modifica date shared). Vrei:

- sa nu intre amandoua in CS in acelasi timp (mutual exclusion)
 - sa nu se blocheze aiurea (progress)
 - sa nu astepte unul la infinit (bounded waiting)
-

2) Ideea de baza (intuitiv)

Fiecare thread:

1. spune “vreau sa intru” (flag)
 2. cedeaza politicos prioritatea celuilalt (turn)
 3. asteapta doar daca si celalalt vrea si e “randul lui”
-

3) Variabilele folosite

Avem 2 variabile shared:

- `flag[2]` (booleene)
 - `flag[i] = true` inseamna: **Pi vrea in CS**
 - `turn` (0 sau 1)
 - `turn = j` inseamna: **daca amandoi vor, il las pe Pj sa intre primul**
-

4) Codul Peterson (standard)

Pentru procesul/thread-ul P_i , unde $j = 1 - i$:

```
// shared
```

```

bool flag[2] = {false, false};

int turn = 0;

// code for process i

flag[i] = true;      // 1) spun ca vreau sa intru

turn = j;            // 2) ii dau prioritate celuiilalt

while(flag[j] && turn == j) {

    ;                // 3) astept (busy waiting)

}

// ---- critical section ----

// ---- exit section ----

flag[i] = false;     // 4) nu mai vreau in CS

// remainder section

```

Scenariul A: Doar P0 vrea sa intre

- P0: flag[0]=true, turn=1
- P1 nu vrea: flag[1]=false

Conditia din while la P0:

- flag[1] && turn==1 => false && ... => false
P0 intra imediat in CS
-

Scenariul B: Amandoi vor sa intre “aproape simultan”

Aici e partea interesanta.

Pas 1: amandoi isi pun flag=true

- P0: flag[0]=true
- P1: flag[1]=true

Pas 2: amandoi seteaza turn

- P0 face `turn = 1`
- P1 face `turn = 0`

Cine seteaza ultimul, decide turn-ul final.

Sa zicem ca **P1 seteaza ultimul**, deci:

- `turn = 0`

Pas 3: verificarea while

- P0 verifica: `flag[1] && turn==1`
 - `flag[1]=true, turn==1?` NU (`turn=0`)
 - `while = true && false = false`
P0 intra in CS
- P1 verifica: `flag[0] && turn==0`
 - `flag[0]=true, turn==0?` DA
 - `while = true && true = true`
P1 asteapta

Pas 4: iesirea lui P0

Cand P0 termina CS:

- P0 face `flag[0]=false`

Acum P1 re-verifica while:

- `flag[0] && turn==0 => false && true => false`
P1 intra

Rezultat: intra unul, apoi celalalt. Niciodata amandoi.

1)Bounded-Buffer (Producer–Consumer)

Problema

- Avem un buffer cu **n** sloturi.
- **Producer** produce iteme si le pune in buffer.
- **Consumer** scoate iteme din buffer.
- Conditii:
 - nu pui daca bufferul e plin
 - nu scoti daca bufferul e gol

- acces exclusiv la buffer (sectiune critica)

Unelte

- `mutex` (semafor binar) = 1 → exclusivitate
- `empty` = n → cate sloturi libere
- `full` = 0 → cate sloturi ocupate

Cod – Producer (pasi)

```
while (true) {  
    produce_item(x);  
  
    wait(empty);    // asteapta slot liber  
  
    wait(mutex);    // intra in sectiunea critica  
  
    add_to_buffer(x);  
  
    signal(mutex); // iese din sectiunea critica  
  
    signal(full);   // anunta ca exista un item  
  
}
```

Cod – Consumer (pasi)

```
while (true) {  
  
    wait(full);     // asteapta item  
  
    wait(mutex);    // intra in sectiunea critica  
  
    x = remove_from_buffer();  
  
    signal(mutex); // iese  
  
    signal(empty);  // elibereaza un slot  
  
    consume_item(x);  
  
}
```

De ce functioneaza

- `empty/full` previn overflow/underflow
- `mutex` previne race condition

Exercitiu tip examen

Intrebare: Ce se intampla daca scoti `mutex`?

Raspuns: Race condition (doi producatori pot scrie in acelasi slot).

2) Readers–Writers

Problema

- Multi **readers** pot citi simultan.
- **Writers** trebuie sa aiba acces exclusiv.
- Variante (prioritati diferite).

Varianta 1: First Readers–Writers (cititorii au prioritate)

Unelte

- `rw_mutex = 1` → control acces la data
- `mutex = 1` → protejeaza `read_count`
- `read_count = 0`

Writer (pasi)

```
while (true) {  
    wait(rw_mutex);  
  
    write_data();  
  
    signal(rw_mutex);  
}
```

Reader (pasi)

```
while (true) {  
    wait(mutex);  
    read_count++;  
    if (read_count == 1)  
        wait(rw_mutex); // primul reader blocheaza writerii  
    signal(mutex);  
    read_data();  
    wait(mutex);  
    read_count--;  
    if (read_count == 0)  
        signal(rw_mutex); // ultimul reader elibereaza  
    signal(mutex);  
}
```

Proprietati

- ✓ multi readers simultan
- ✗ writer starvation posibil

Exercitiu tip examen

Intrebare: De ce poate aparea starvation la writers?

Raspuns: Readers noi pot intra continuu si nu mai elibereaza rw_mutex.

3) Dining Philosophers

Problema

- N filosofi, N betisoare.
- Fiecare are nevoie de **doua betisoare** ca sa manance.

- Risc: **deadlock** si **starvation**.

Solutia simpla cu semafoare (PROBLEMATICA)

```
while (true) {
    wait(chopstick[i]);
    wait(chopstick[(i+1)%N]);
    eat();
    signal(chopstick[i]);
    signal(chopstick[(i+1)%N]);
    think();
}
```

Problema

- Deadlock: toti iau betisorul din stanga si asteapta dreapta.

Solutie corecta cu Monitor

Ideea

- Monitorul decide cine poate manca.
- Starile: THINKING, HUNGRY, EATING.
- Un filosof mananca doar daca vecinii NU mananca.

Functii cheie

```
pickup(i):
    state[i] = HUNGRY;
    test(i);
    if (state[i] != EATING)
        wait(self[i]);
putdown(i):
```

```
state[i] = THINKING;

test(left(i));

test(right(i));
```

test(i)

```
if (left not EATING &&

    state[i] == HUNGRY &&

    right not EATING) {

    state[i] = EATING;

    signal(self[i]);

}
```

Proprietati

- ✓ fara deadlock
- ✗ starvation posibil (in unele cazuri)

Exercitiu tip examen

Intrebare: De ce nu exista deadlock?

Raspuns: Monitorul permite acces doar cand ambii vecini nu mananca.

4) Sincronizare cu TAS si CAS

TAS (Test-And-Set)

- Atomica
- Creeaza **spinlock**
- NU garanteaza bounded waiting

```
while (test_and_set(&lock))
```

```
;
```

```
/* critical section */
```

```
lock = false;
```

CAS (Compare-And-Swap)

- Atomica
- Mai flexibila
- Poate construi algoritmi lock-free

```
while (compare_and_swap(&lock, 0, 1) != 0)
```

```
;
```

```
/* critical section */
```

```
lock = 0;
```

CAS cu bounded waiting

- Foloseste `waiting[ ]`
- Asigura echitate (fara starvation)

5) Intrebari scurte de examen (cu raspuns)

1. **Ce problema rezolva bounded-buffer?**
→ Sincronizarea producator–consumator cu buffer finit.
2. **Ce conditie rupe mutex-ul?**
→ Mutual exclusion.
3. **De ce CAS este hardware?**
→ CPU garanteaza atomicitatea intr-o singura instructiune.
4. **De ce semafoarele pot cauza deadlock?**
→ Ordine gresita de `wait()` pe resurse.

MAIN MEMORY

1) Notiuni de baza

Logical address vs Physical address

- **Logical (virtual):** adresa generata de CPU, vazuta de proces
- **Physical:** adresa reala din RAM

Traducerea se face de MMU (Memory Management Unit)

De ce avem memory management

- RAM este limitata
 - mai multe procese ruleaza simultan
 - un proces NU trebuie sa acceseze memoria altuia
 - vrem eficienta + protectie
-

2) Base & Limit (relocation + protection)

- **base (relocation)**: adresa fizica de start
- **limit**: dimensiunea maxima permisa

Regula

daca `logical_address >= limit` → TRAP (eroare)

altfel `physical = base + logical`

EXEMPLU

- base = 30000
- limit = 1000

Adresa logica	Rezultat
200	30200
999	30999
1200	TRAP

Protectie simpla, dar **nu flexibila**

3) Address Binding (cand se leaga adresele)

Tipuri

1. **Compile time** – adrese fixe (rar)
2. **Load time** – adrese relocate la incarcare
3. **Execution time** – adrese mutate in timpul rularii (paging)
Paging = execution-time binding

4) Paging – conceptul CHEIE

Definitii

- **Page** = bucata fixa din memoria logica (proces)
- **Frame** = bucata fixa din RAM
- Page size = frame size

Paging elimina **external fragmentation**

5) Structura adresei logice (p | d)

Daca:

- adresa logica are **m biti**
- page size = 2^n

Atunci:

- **offset (d)** = n biti
- **page number (p)** = m - n biti
- **nr pagini** = $2^{(m-n)}$

EXEMPLU 1 (foarte tipic)

- CPU pe 16 biti
- page size = 4 KB = 2^{12}
- d = 12 biti
- p = 4 biti
- nr pagini = 16

6) Address Translation

PASII

1. CPU genereaza adresa logica
2. separi p si d
3. $\text{page_table}[p] \rightarrow \text{frame}$
4. $\text{physical} = \text{frame} * \text{page_size} + d$

EXEMPLU 2

- page size = 1024 bytes
- $\text{page_table}[3] = \text{frame } 7$
- adresa logica: p=3, d=100

$$\text{physical} = 7 * 1024 + 100 = \mathbf{7268}$$

7) Page Table (ce contine)

Un entry poate contine:

- frame number
- valid / invalid bit
- protection bits (R/W/X)
- dirty bit (modificata sau nu)
- reference bit

Valid / Invalid bit

- **valid** $\rightarrow$ pagina apartine procesului
- **invalid** $\rightarrow$ acces ilegal $\rightarrow$ TRAP

8) Internal Fragmentation

Formula: $\text{internal_frag} = \text{page_size} - (\text{process_size} \bmod \text{page_size})$

EXEMPLU 3

- page size = 2048 bytes
- process size = 72766 bytes
- 35 pagini + 1086 bytes
- $\text{internal frag} = 2048 - 1086 = \mathbf{962 \text{ bytes}}$

worst case ≈ 1 page

average $\approx 1/2$ page

9) Page Table in RAM + problema celor 2 accese

Fara TLB:

- 1 acces → page table
 - 1 acces → data
 - 2 accesari de memorie pentru 1 instructiune
-

10) TLB (Translation Lookaside Buffer)

Ce este

- cache hardware mic
- tine mapari pagina → frame

Avantaj

- daca e **hit** → 1 acces
 - daca e **miss** → 2 accese
-

EAT – Effective Access Time

Formula

$$EAT = h * M + (1 - h) * 2M$$

unde:

- h = hit ratio
- M = timp acces memorie

EXEMPLU 4

- M = 10 ns
 - h = 0.80
- $$EAT = 0.8 \cdot 10 + 0.2 \cdot 20 = 8 + 4 = 12 \text{ ns}$$

Cu h = 0.99:

$$EAT \approx 10.1 \text{ ns}$$

11) Shared Pages

- cod read-only poate fi share-uit între procese
- datele rămân private

Ex: librării shared (printf, math)

12) Structura page table (de ce devine mare)

Exemplu 32-bit

- page size = 4 KB
 - 2^{20} pagini
 - 4 bytes / entry
page table = **4 MB / proces**
-

13) Two-Level Paging

Spargerea adresei

| p1 | p2 | d |

PASII

1. p1 → outer page table
2. p2 → inner page table
3. obții frame
4. adaugi d

economisești memorie (aloci inner table doar dacă e folosită)

14) Swapping

Ce este

- mutarea unui proces din RAM pe disk
- eliberezi RAM pentru alte procese

Important

- swap time $\propto$ cantitatea de date
- foarte lent comparat cu RAM

EXEMPLU 5

- proces = 100 MB
- disk = 50 MB/s

swap out = 2s

swap in = 2s

total = **4 secunde**

15) Comparatii IMPORTANT DE STIUT

Paging vs Contiguous

Paging	Contiguous
fara external frag	external frag
management mai complex	simplicu
foloseste page table	base + limit

- $\text{physical} = \text{frame} * \text{page_size} + d$
 - $\text{page_size} = 2^{\text{offset_bits}}$
 - $p_bits = m - n$
 - $\text{internal frag} \approx \text{page_size} - \text{rest}$
 - $\text{EAT} = h * M + (1-h) * 2M$
 - invalid bit $\rightarrow$ TRAP
-

CURS 10: VIRTUAL MEMORY

1) Virtual Memory – ideea de baza (teorie scurta)

Virtual memory permite unui proces sa aiba un spatiu de adrese mai mare decat memoria RAM fizica.

Doar paginile necesare sunt incarcate in RAM, restul raman pe disk.

Avantaje:

- mai multe procese simultan
- consum mai mic de RAM
- mai putin I/O
- pornire rapida a programelor
- sharing intre procese

2) Virtual Address Space

Structura tipica:

- code (read-only)
- data
- heap (creste in sus)
- hole (spatiu liber)
- stack (creste in jos)

Pagini din acest spatiu sunt aduse in RAM doar cand sunt accesate.

3) Demand Paging

Definitie: Paginiile sunt incarcate in RAM doar la prima accesare.

Ce se intampla la acces normal

- pagina este deja in RAM
- accesul continua normal

Ce se intampla la acces absent

- pagina NU este in RAM
- apare **page fault**

4) Page Fault – PASII OBLIGATORII

Cand CPU acceseaza o pagina absenta:

1. CPU face trap in OS

2. OS verifica daca accesul e legal
 - ilegal → abort / segfault
3. OS gaseste un frame liber
4. OS citeste pagina de pe disk in RAM
5. OS actualizeaza page table (valid = 1)
6. OS reia instructiunea

Exemplu simplu

Procesul acceseaza pagina 5:

- PT[5] = invalid
- page fault
- OS o aduce in frame 12
- PT[5] = 12, valid
- instructiunea se reexecuta

5) Performanta Demand Paging

Disk este mult mai lent decat RAM (ms vs ns).

Chiar si: 1 page fault la 1000 accesari
duce la scadere mare de performanta.

Concluzie: rata page fault-urilor trebuie sa fie FOARTE mica.

6) Copy-on-Write (COW)

La fork():

- parinte si copil share-uiesc aceleasi pagini
- paginile sunt read-only

Cand apare copierea

- doar cand unul scrie intr-o pagina
- OS creeaza o copie noua doar pentru acel proces

Exemplu

- parinte si copil share pagina A
- copil scrie in A
- OS copiaza A doar pentru copil

Avantaj:

- fork rapid
- memorie economisita

7) Cand nu mai ai frame-uri libere

Daca apare page fault si: NU exista frame liber
=> trebuie **page replacement**

8) Page Replacement – PASII COMPLETI

1. apare page fault
2. alegi o pagina victima (algoritm)
3. daca victima e dirty: write pe disk
4. incarci pagina noua
5. update page table
6. restart instruction

9) Dirty bit

- dirty = 0 → pagina nu a fost modificata → o poti arunca
- dirty = 1 → pagina a fost modificata → trebuie scrisa pe disk

10) PAGE REPLACEMENT ALGORITHMS (CU PASI + EXEMPLE)

A) FIFO (First-In First-Out)

Scoti pagina care a intrat prima.

Pasi

1. tii o coada
2. la fault si plin: scoti prima intrata
3. adaugi pagina noua la final

Exemplu

Reference: 1 2 3 1 4

Frames = 3

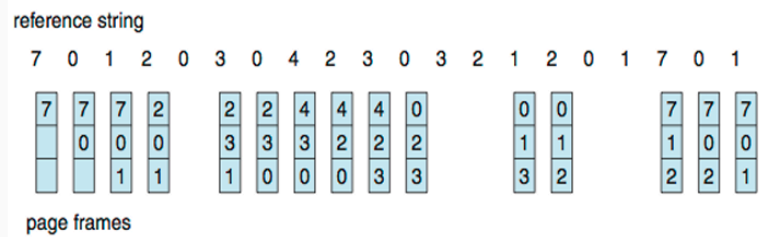
- 1 → [1] fault
- 2 → [1 2] fault
- 3 → [1 2 3] fault
- 1 → hit

- 4 → scoti 1 → [4 2 3] fault

Total faults = 4

Ex curs

- Reference string: **7,0,1,2,0,3,0,4,2,3,0,3,2,1,2,0,1,7,0,1**
- 3 frames (3 pages can be in memory at a time per process)



15 page faults

FIFO poate avea **Belady anomaly**

B) OPT (Optimal)

Scoti pagina care va fi folosita cel mai tarziu in viitor.

Pasi

1. la fault:
2. te uiti in viitor pentru fiecare pagina
3. scoti pagina cu utilizare cea mai indepartata

Exemplu

Frames = [1 2 3], vine 4

Viitor:

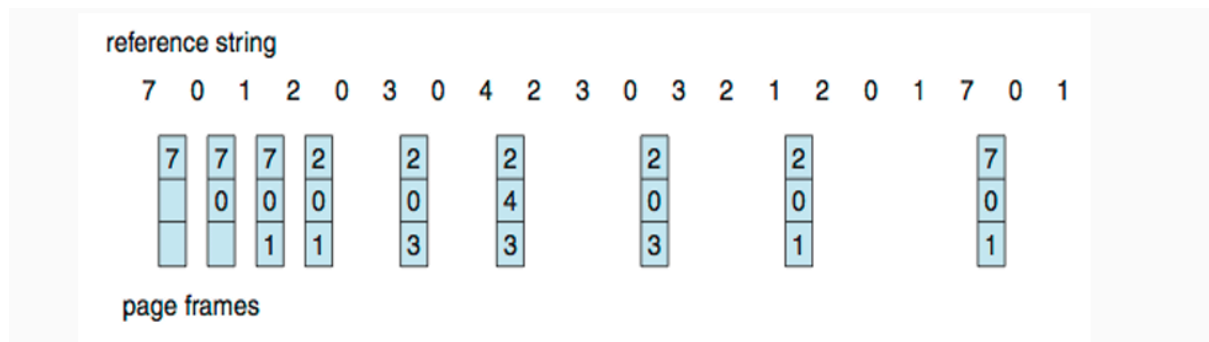
- 1 → nu mai apare
- 2 → nu mai apare
- 3 → nu mai apare

Scoti oricare (ex: 1)

algorithm ideal

nu se poate implementa real

Ex curs



C) LRU (Least Recently Used)

Scoti pagina nefolosita de cel mai mult timp (in trecut).

Pasi

1. tii minte ultima utilizare
2. la fault: scoti cea mai veche folosita

Exemplu

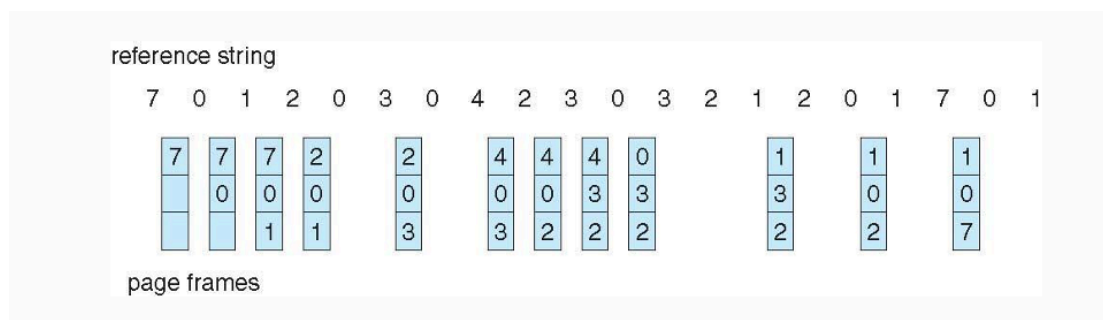
Reference: 1 2 3 1 4 , Frames = [1 2 3]

Ultima folosire:

- 1 → tocmai
- 2 → cel mai vechi
- 3 → mai recent

Scoti 2 → [1 4 3]

Ex curs



D) Second Chance / CLOCK

FIFO + reference bit R.

Pasi

1. pointer circular
2. la fault:
 - daca $R=0 \rightarrow$ scoti
 - daca $R=1 \rightarrow$ setezi 0 si sari

Exemplu

Frames: 1 ($R=1$) 2 ($R=1$) 3 ($R=0$)

Vine 4:

- $1 \rightarrow R=0$
- $2 \rightarrow R=0$
- $3 \rightarrow$ scos

Frames finale: 1 2 4

E) Enhanced Second Chance

Foloseste (R, M):

Ordine de scos:

1. (0,0)
 2. (0,1)
 3. (1,0)
 4. (1,1)
-

F) LFU / MFU

- LFU: scoti pagina cu cele mai putine accesari
- MFU: scoti pagina cu cele mai multe accesari

Rar folosite.

11) Allocation of Frames

- equal allocation
- proportional allocation
- priority allocation

12) Global vs Local Replacement

- global: procesele pot lua frame-uri intre ele
- local: fiecare doar in frame-urile lui

13) Thrashing

Definitie

Procesul face page fault-uri continuu si petrece mai mult timp in swap decat in executie.

Cauza: prea putine frames pentru working set

Solutii

- reduce multiprogramming
- mai multe frames
- local replacement

14) Locality

Procesele au:

- localitate temporala
- localitate spatiala

Explica de ce demand paging functioneaza.

15) Working Set Model

- Δ = fereastra
- WSS = pagini folosite in Δ
- daca suma WSS > frames $\rightarrow$ thrashing

16) Page Fault Frequency (PFF)

- rata mare $\rightarrow$ dai frames
- rata mica $\rightarrow$ iei frames

17) Memory-Mapped Files

- fisier mapat in spatiul virtual
- acces ca la memorie
- paging aduce pagini

18) Kernel Memory

Buddy System

- aloca in puteri ale lui 2
- fragmentare interna

Slab Allocator

- cache per tip de obiect
- rapid, eficient

19) Capcane

- page fault $\neq$ TLB miss
- TLB nu contine pagini de pe disk
- FIFO poate avea mai multe frames $\rightarrow$ mai multe faults
- thrashing $\neq$ deadlock
- COW copiaza doar la scriere

MINI-RETETE DE MEMORAT

- Demand paging: fault $\rightarrow$ aduci $\rightarrow$ update $\rightarrow$ restart
- Replacement: fault + plin $\rightarrow$ victima
- COW: share $\rightarrow$ write $\rightarrow$ copy
- Thrashing: WSS > frames

FILE SYSTEM INTERFACE

Un **fisier** este:

- o **zona logica continua de date**
- stocata pe disc
- vazuta de utilizator ca o unitate (nume + continut)

Tipuri de fisiere

- text (ex: .txt)

- sursa (ex: .c)
- binar (ex: executabile)
- date (ex: .csv, .dat)

OS NU interpreteaza continutul, doar il gestioneaza.

Atributele unui fisier (File Attributes)

Fiecare fisier are:

- **Name** – numele
- **Identifier** – ID unic (inode)
- **Type** – tipul fisierului
- **Location** – unde e pe disc
- **Size** – dimensiunea
- **Protection** – cine are voie (read/write/execute)
- **Time/Date/User** – creare, modificare, acces

Toate sunt tinute in **director**.

Operatii pe fisiere (File Operations)

Operatii standard: create, open, read, write, seek (mutare cursor), close, delete, truncate

Exemplu logic:

```
open("a.txt")
```

```
read()
```

```
write()
```

```
close()
```

Open File Table

Cand un fisier e deschis, OS foloseste:

- **tabela globala de fisiere deschise**
- **tabela per proces**

Ce se tine:

- file pointer (pozitia curenta)
- numar de procese care l-au deschis
- drepturi de acces
- locatie pe disc

De aceea:

- doua procese pot citi acelasi fisier
- fiecare are **cursor propriu**

File Locking (blocare fisiere)

Similar cu **readers-writers**.

Tipuri:

- **Shared lock** – mai multi cititori
- **Exclusive lock** – un singur scriitor

Mandatory vs Advisory

- **Mandatory** – OS impune blocarea
- **Advisory** – procesele decid singure

Foarte des apare la examen ca analogie cu RW-problem.

Structura fisierului (File Structure)

Cine decide structura?

- OS sau
- programul

Tipuri:

- fara structura (sir de bytes)
- linii
- inregistrari (records)
- structuri complexe

UNIX: fisier = **sir de bytes**

Metode de acces (Access Methods)

7.1 Sequential Access

- citești **in ordine**
- read next, write next
- nu sari usor

Exemple: fisiere text

7.2 Direct Access

- citești **pozitia n**
- acces random

Exemplu:

read block 5

write block 10

7.3 Indexed Access

- foloseste un **index**
- cautare rapida

Exemplu: baze de date

Structura discului (Disk Structure)

Discul este impartit in:

- **partitii**
- fiecare partitie = **volum**
- volum → sistem de fisiere

Un disc poate avea:

- mai multe sisteme de fisiere
- sisteme speciale (tmpfs, procfs)

Structura directoarelor (Directory Structure)

Director = fisier special care contine:

- nume fisier
- pointer catre inode

Operatii: search, create, delete, rename, traverse

Organizarea directoarelor

a) Single-level

- un singur director: conflicte de nume

b) Two-level

- director per utilizator
- /user/file

c) Tree (cel mai folosit)

- ierarhie
- cai absolute/relative

UNIX foloseste **tree structure**

Memory-Mapped Files (important!)

Fisierul este:

- mapat direct in **memorie**
- accesat ca un array

Avantaje:

- mai rapid
- foloseste demand paging

Legatura directa cu capitolul de **memorie virtuala**.

Protectia fisierelor

Control acces: read, write, execute

Pe: owner, group, others
Analog cu protectia memoriei.

Asociere cu alte capitole (foarte apreciat)

- **Locks** ↔ sincronizare
- **Memory-mapped files** ↔ paging
- **Open-file table** ↔ procese
- **File locking** ↔ readers–writers

FILE SYSTEM IMPLEMENTATION

1. File-System Structure (Structura sistemului de fişiere)

- Se află pe secondary storage (disk)
- Oferă o interfaţă între:
 - fişiere logice
 - blocuri fizice pe disk
- Permite:
 - stocare
 - localizare
 - acces eficient la date

1.2 File Control Block (FCB)

- Pentru fiecare fişier există un FCB (în UNIX: inode)
- Conţine:
 - permissions
 - owner / group
 - size
 - date (create / access / modify)
 - pointeri către blocurile de date

Notă: Directorul nu conţine datele fişierului, ci pointeri către FCB.

2. Layered File System (foarte important)

Sistemul de fişiere este organizat pe straturi:

1. Application programs
2. Logical file system

- metadata
- directoare
- protecție
- 3. File-organization module
 - traducere blocuri logice → fizice
 - alocare spațiu
- 4. Basic file system
 - citire / scriere blocuri
 - buffer cache
- 5. I/O control
 - drivere
- 6. Devices

Avantaj: modularitate

Dezavantaj: overhead

3. File-System Operations (structuri on-disk și in-memory)

3.1 Structuri pe disk

- Boot control block
 - informații pentru boot (dacă discul conține OS)
- Volume control block (superblock)
 - număr total de blocuri
 - număr blocuri libere
 - dimensiune bloc
- Directory structure
 - nume fișier + inode number
- Data blocks
 - conținutul efectiv al fișierelor

3.2 Structuri în memorie

- Mount table
- System-wide open-file table
- Per-process open-file table
- Buffer cache

4. Directory Implementation

4.1 Linear list

- Listă de perechi (nume, pointer)

Avantaje: implementare simplă

Dezavantaje: căutare lentă $O(n)$

Optimizări:

- listă ordonată
- B+ tree

4.2 Hash table

- nume $\rightarrow$ funcție hash $\rightarrow$ locație

Avantaje: căutare rapidă

Dezavantaje: coliziuni, dificil de gestionat intrări de dimensiune variabilă

5. Allocation Methods (subiect clasic)

5.1 Contiguous allocation

- Fișierul ocupă blocuri consecutive pe disk
- În director se rețin:
 - start block
 - length

Avantaje: acces foarte rapid (sequential și random)

Dezavantaje: fragmentare externă, dimensiunea fișierului trebuie cunoscută dinainte

Exemplu: Fișierul începe la blocul 10 și are lungimea 4 $\rightarrow$ blocurile 10, 11, 12, 13

5.2 Extent-based allocation

- Compromis modern
- Fișierul este format din mai multe extents
- Fiecare extent conține blocuri consecutive

Avantaje: reduce fragmentarea, performanță bună

Notă: utilizată în file system-uri moderne

5.3 Linked allocation

- Fișierul este o listă înlănțuită de blocuri
- Fiecare bloc conține pointer către următorul

Avantaje: fără fragmentare externă

Dezavantaje: acces random lent, risc ridicat la coruperea pointerilor

Exemplu: $9 \rightarrow 13 \rightarrow 6 \rightarrow 21 \rightarrow \text{EOF}$

5.4 File Allocation Table (FAT)

- Tabela FAT păstrează legăturile dintre blocuri
- Fiecare intrare indică următorul bloc al fișierului

Avantaje: pointerii nu sunt stocați în blocurile de date, acces mai rapid decât linked allocation simplu

Dezavantaje: FAT trebuie păstrată în memorie și poate fi mare

Notă: utilizată în FAT, FAT32

5.5 Indexed allocation

- Fiecare fișier are un index block
- Index block-ul conține pointeri către blocurile de date

Avantaje: acces direct rapid

Dezavantaje: overhead suplimentar pentru index block

Exemplu: Index block 19 $\rightarrow$ [9, 16, 1, 10, 25]

6. Free-Space Management (foarte important)

6.1 Bit map (bit vector)

- Un bit per bloc
- 1 = liber, 0 = ocupat

Avantaje: ușor de găsit spațiu contiguu

Dezavantaje: consum mare de memorie pentru disk-uri mari

6.2 Linked free list

- Blocurile libere sunt organizate într-o listă

Avantaje: fără risipă de spațiu

Dezavantaje: dificil de găsit spațiu contiguu

6.3 Grouping

- Un bloc conține adresele mai multor blocuri libere

Avantaje: reduce numărul de accesări la disk

6.4 Counting

- Se memorează:
 - bloc de start
 - număr de blocuri libere consecutive

Avantaje: foarte eficient pentru extent-based allocation

6.5 Space maps (ZFS)

- utilizarea metaslabs
- jurnalizare
- structuri de tip balanced tree

7. TRIM (SSD / NVM)

- HDD: overwrite direct
- SSD/NVM: este necesară operația de erase înainte de write

TRIM:

- file system-ul informează dispozitivul ce blocuri sunt libere
- dispozitivul le poate șterge în avans

Avantaj: performanță îmbunătățită

8. Efficiency and Performance

Factori:

- metoda de alocare
- structura directoarelor
- buffering
- caching

Tehnici:

- buffer cache
- read-ahead
- free-behind
- asynchronous writes

9. Page Cache vs Buffer Cache

Page cache: folosit pentru memory-mapped files

Buffer cache: folosit pentru operații clasice de citire/scriere

Unified buffer cache: un singur cache pentru ambele

Avantaj: evită dublarea datelor

Dezavantaj: politici de replacement mai complexe

10. Recovery (foarte important)

10.1 Consistency checking

- compară:
 - directoare
 - FCB
 - blocuri

- poate fi lent

10.2 Journaling / Log-Structured File Systems

- fiecare operație este o tranzacție
- tranzacția este scrisă mai întâi în jurnal
- apoi este aplicată efectiv

Avantaje: recovery rapid, evită inconsistențele

Exemple: ext4, NTFS, APFS

11. WAFL File System

Write-Anywhere File Layout:

- folosește inode-uri
- optimizat pentru operații de scriere
- suportă snapshots

Snapshots

- copie logică a sistemului de fișiere
- copy-on-write
- blocurile nemodificate sunt partajate

Recapitulare rapidă

- FCB = metadata fișier
- Layered FS = logical → physical
- Allocation = contiguous, linked, FAT, indexed
- Free space = bitmap, linked list, counting
- Journaling = recovery rapid
- Unified buffer cache = un singur cache